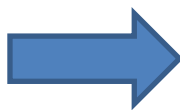




Introducción a la Programación

Trabajo Práctico: Visualización de Algoritmos de Ordenamiento



Alumnos:

Encinas Ezequiel

Guido Vazquez

Docentes:

Fernando Velcic

Patricia Bagnes

Universidad Nacional General Sarmiento

Introducción a la Programación

Comisión 05

17 de Noviembre del año 2025

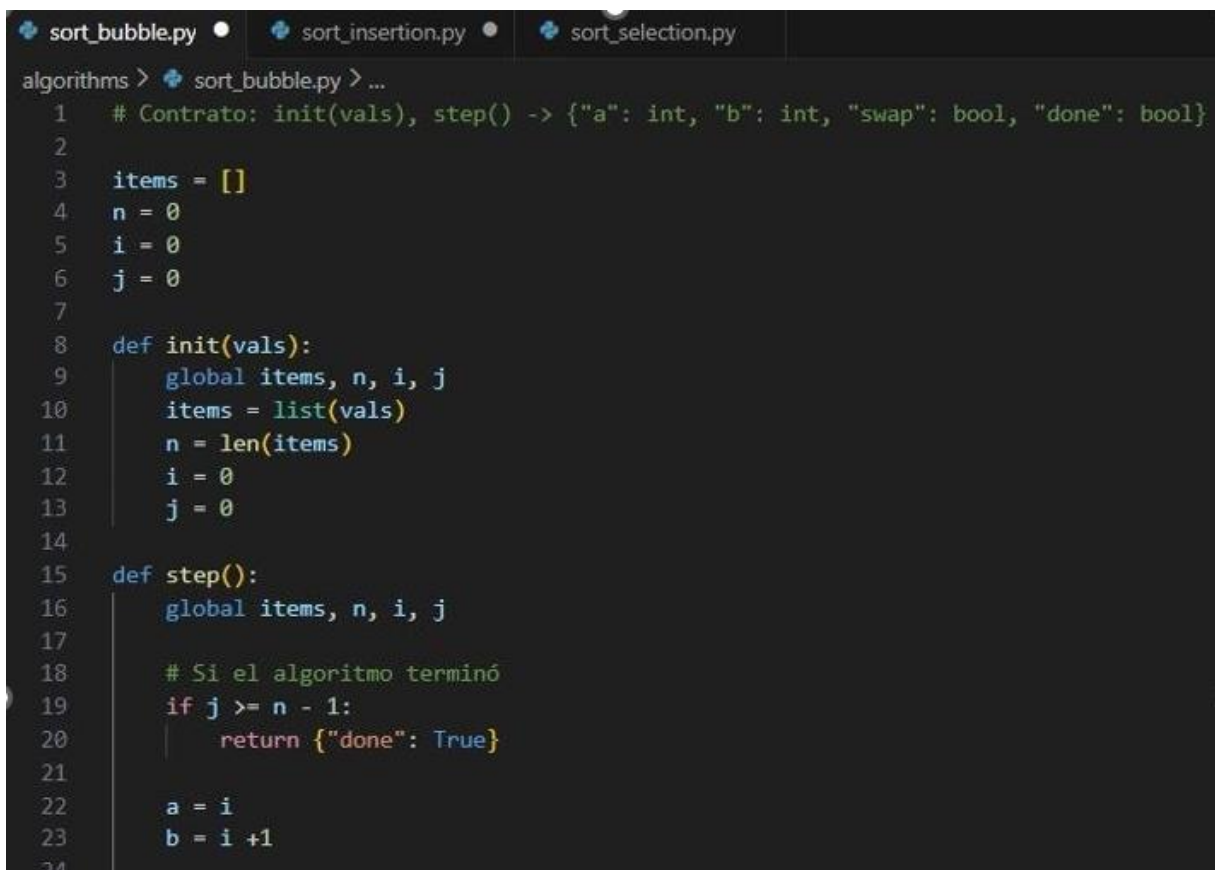
Introducción:

En este trabajo analizamos y comprobamos cómo funcionan tres tipos distintos de algoritmos para ordenar (*Bubble*, *Insertion*, *Selection*). Para esto programamos cada método por separado en Python con For o While, para luego poder probarlos en un visualizador web que muestra paso a paso cómo se van acomodando los valores. El objetivo del trabajo fue comparar sus comportamientos y entender qué decisiones toma cada uno y cómo se comportan para llegar al orden final.

Explicación del Bubble Sort:

El método de ordenamiento burbuja o por burbujeo, ordena comparando elementos que están uno al lado del otro. Los compara, y si uno está en el lugar incorrecto los cambia de lugar. Repite este proceso varias veces, empujando la lista hacia la derecha de principio a final, hasta que quedan los elementos ordenados. Una dificultad que apareció fue adaptar el código original, hecho en Python, al entorno del visualizador, porque allí ya existían ciertos ciclos y variables predefinidos. Tuvimos que ajustar la estructura del algoritmo para que siguiera funcionando dentro de esa primera idea que surgió sin modificar su comportamiento.

Código del algoritmo Bubble Sort:



```
sort_bubble.py • sort_insertion.py • sort_selection.py
algorithms > sort_bubble.py > ...
1  # Contrato: init(vals), step() -> {"a": int, "b": int, "swap": bool, "done": bool}
2
3  items = []
4  n = 0
5  i = 0
6  j = 0
7
8  def init(vals):
9      global items, n, i, j
10     items = list(vals)
11     n = len(items)
12     i = 0
13     j = 0
14
15  def step():
16      global items, n, i, j
17
18      # Si el algoritmo terminó
19      if j >= n - 1:
20          return {"done": True}
21
22      a = i
23      b = i + 1
24
```

```

24
25     # Comparar e intercambiar si corresponde
26     swap = False
27     if items[a] > items[b]:
28         aux = items[a]
29         items[a] = items[b]
30         items[b] = aux
31         swap = True
32
33     # Avanzar punteros
34     i = i + 1
35     if i >= n - 1 - j: # Llego al final de la pasada
36         i = 0
37         j = j + 1
38     return {"a": a, "b": b, "swap": swap, "done": False}
39

```

Explicación del Selection Sort:

El método de ordenamiento por selección busca en cada pasada el elemento más chico de la parte que queda por ordenar y lo coloca en su posición correcta. Repite este procedimiento avanzando paso a paso por la lista hasta completar el ordenamiento.

En el proceso de implementación tuvimos que adaptar nuestra lógica con la estructura que traía el visualizador, que ya incluía ciertos pasos del recorrido. Para esto tuvimos que reorganizar el algoritmo para que utilizara las variables y los ciclos ya establecidos sin alterar su funcionamiento

Código del algoritmo Selection Sort:

algorithms > sort_selection.py > step

```
1  items = []
2  n = 0
3  i = 0      # cabeza de la parte no ordenada
4  j = 0      # cursor que recorre y busca el mínimo
5  min_idx = 0 # índice del mínimo de la pasada actual
6  fase = "buscar" # "buscar" | "swap"
7
8  def init(vals):
9      global items, n, i, j, min_idx, fase
10     items = list(vals)
11     n = len(items)
12     i = 0
13     j = i + 1
14     min_idx = i
15     fase = "buscar"
16
17  def step():
18     global items, n, i, j, min_idx, fase
19
20     # Si ya terminamos
21     if i >= n - 1:
22         return {"done": True}
23
24     # Fase de buscar el mínimo
25     if fase == "buscar":
26         if j < n:      # Mientras j esté dentro de la lista, seguimos comparando
27             a = min_idx # Índice del mínimo actual
28             b = j        # Índice que estamos comparando ahora
29
```

```
30         # Comparacion
31         if items[j] < items[min_idx]:
32             min_idx = j # Nuevo mínimo encontrado
33
34         j += 1 # Avanzar a la siguiente vuelta
35
36         return {"a": a, "b": b, "swap": False, "done": False}
37
38     # Si terminamos de desplazarlos, pasamos a la fase swap
39     fase = "swap"
40
41
42     # Fase de hacer el swap
43     if fase == "swap":
44         a = i
45         b = min_idx
46
47         hubo_swap = False
48
```

```
49     if min_idx != i:
50         # Hacemos el swap real en items
51         aux = items[i]
52         items[i] = items[min_idx]
53         items[min_idx] = aux
54         hubo_swap = True
55
56     # Avanzar a la próxima pasada
57     i = i + 1
58     j = i + 1
59     min_idx = i
60     fase = "buscar"
61
62     return { "a": a, "b": b, "swap": hubo_swap, "done": False}
```

Explicación del algoritmo Insertion Sort:

Este método ordena la lista como si se estuvieran acomodando cartas en la mano: va tomando un elemento y lo coloca en el lugar que le corresponde respecto a los anteriores. Para lograrlo, va corriendo los elementos hacia un costado hasta encontrar el espacio adecuado.

Al principio pensamos que este método intercambiaba valores directamente, cuando en realidad lo que hace es mover posiciones dentro de la lista. Esta confusión nos hizo replantear su estructura y volver a analizar paso a paso cómo avanza el ordenamiento. Una vez aclarado esto, pudimos adaptarlo al entorno del visualizador sin problemas.

Código del algoritmo Insertion Sort:

```
algorithms > sort_insertion.py > step
1  # Contrato: init(vals), step() -> {"a": int, "b": int, "swap": bool, "done": bool}
2
3  items = []
4  n = 0
5  i = 0      # elemento que queremos insertar
6  j = None   # cursor de desplazamiento hacia la izquierda (None = empezar)
7
8  def init(vals):
9      global items, n, i, j
10     items = list(vals)
11     n = len(items)
12     i = 1    # común: arrancar en el segundo elemento
13     j = None
14
15  def step():
16     global items, n, i, j
17
18     # Si ya terminamos
19     if i >= n:
20         return {"done": True}
21
22     # Si j es None, iniciamos el desplazamiento del elemento en i
```

```
21
22     # Si j es None, iniciamos el desplazamiento del elemento en i
23     if j == None:
24         j = i
25         return {"a": j-1, "b": j, "swap": False, "done": False}
26
27     if j > 0 and items[j-1] > items[j]: # Si podemos desplazar hacia la izquierda
28         # Hacemos swap
29         aux = items[j-1]
30         items[j-1] = items[j]
31         items[j] = aux
32         j = j - 1
33         return {"a": j, "b": j+1, "swap": True, "done": False}
34
35     # Ya no hay que desplazar entonces avanzamos al siguiente i
36     i = i + 1
37     j = None
38     return {"a": -1, "b": -1, "swap": False, "done": False}
```

Conclusión:

A lo largo del trabajo práctico, y al analizar cada uno de los algoritmos de ordenamiento, pudimos identificar sus características y diferencias de funcionamiento. El uso del visualizador resultó muy útil para observar de manera dinámica el comportamiento de los algoritmos en distintas velocidades y paso por paso, lo que facilitó su comprensión y entendimiento. Además todos los ejercicios vistos y realizados en las clases aportaron una buena base para alcanzar el objetivo. Sin dudas fue un trabajo desafiante, que en equipo pudimos lograr el resultado esperado